

TP Teórico - Grupo 1

Introducción a Ruby	2
Características de Ruby.....	2
BNF de RUBY (Resumida).....	3
Explicación.....	3
Introducción a Go.....	5
Características de Go.....	5
BNF de GO.....	6
Explicación.....	6
Algoritmos de Ordenamiento.....	7
Ruby: La flexibilidad y el enfoque híbrido (Array#sort).....	7
Go: La fuerza bruta inteligente (pdqsort).....	8
¿Qué es .sort? (Interfaz vs. Implementación).....	8
Los Algoritmos "Puros" (La base teórica).....	8
• QuickSort (Ordenamiento Rápido):.....	8
• MergeSort (Ordenamiento por Mezcla):.....	9
• Insertion Sort (Ordenamiento por Inserción):.....	9
Los Algoritmos "Híbridos" (El mundo real de Ruby y Go).....	9
La Implementación en Go: pdqsort (Pattern-Defeating Quicksort).....	9
La Implementación en Ruby: IntroSort y TimSort.....	9
Resumen Comparativo: Ruby vs. Go (Ordenamiento).....	10
Ejemplo Ordenamiento en Ruby.....	11
Ejemplo Ordenamiento en Go.....	13
Fuentes:.....	15

Introducción a Ruby

Ruby fue concebido a partir del año 1993 y desarrollado a mediados de la década de 1990 en Japón.

El creador de este lenguaje es Yukihiro "Matz" Matsumoto.

La primera versión pública del lenguaje (Ruby 0.95) fue anunciada y lanzada en diciembre de 1995.

La principal motivación de Matsumoto era diseñar un lenguaje enfocado en la productividad, la simplicidad y la felicidad del programador, logrando que la programación fuera una tarea más agradable y humana.

Para lograrlo, buscó crear un lenguaje que equilibrara la programación funcional y la imperativa, tomando una fuerte inspiración de lenguajes previos como Perl, Smalltalk, Eiffel, Ada, BASIC y Lisp. Como dato curioso, el nombre "Ruby" (Rubí) surgió en 1993 durante un chat en línea antes de que se escribiera el código, y Matsumoto lo eligió porque era la piedra de nacimiento de uno de sus colegas.

Características de Ruby

Orientado a objetos puro: En Ruby, absolutamente todos los valores son tratados como un objeto. Esto significa que incluso los tipos de datos primitivos, como los números enteros o el valor nulo (nil), poseen atributos y métodos propios.

Multiparadigma: Es un lenguaje sumamente versátil que soporta múltiples paradigmas, permitiendo el uso de programación orientada a objetos, programación procedimental (imperativa) y programación funcional.

Tipado Dinámico: Ruby es un lenguaje dinámico, por lo cual no exige que se declare el tipo de una variable en el momento de crearla.

Gestión Automática de Memoria: Posee un recolector de basura (garbage collector) integrado que se encarga de gestionar automáticamente el manejo de la memoria de todos los objetos, facilitando el desarrollo.

Sintaxis Intuitiva: El lenguaje destaca por tener una sintaxis elegante, humana y natural que permite a los desarrolladores expresar sus ideas de manera concisa y modular.

Alta Portabilidad: Aunque originalmente fue desarrollado para el entorno GNU/Linux, Ruby es altamente portable y puede ejecutarse en distintos sistemas operativos como UNIX, Windows, MacOS y DOS.

BNF de RUBY (Resumida)

```
<programa>          ::= <lista_expr>
<lista_expr>         ::= <expr> [ <terminador> <lista_expr> ]
<expr>               ::= <asignacion> | <llamada_metodo> | <estructura_control> |
                        <literal> | <def_clase> | <def_metodo> | <identificador>
<def_clase>          ::= "class" <Constante> [ "<" <Constante> ] <terminador>
                        <lista_expr>
                        "end"
<def_metodo>         ::= "def" <identificador> [ "(" <lista_params> ")" ] <terminador>
                        <lista_expr>
                        "end"
<lista_params>       ::= <identificador> [ "," <lista_params> ] | ε
<estructura_control> ::= "if" <expr> [ "then" ] <terminador>
                        <lista_expr>
                        [ "elsif" <expr> [ "then" ] <terminador> <lista_expr> ]*
                        [ "else" <terminador> <lista_expr> ]
                        "end"
<asignacion>         ::= <identificador> "=" <expr>
<terminador>        ::= ";" | "\n"
```

BNF de Ruby: <https://docs.huihoo.com/ruby/ruby-man-1.4/yacc.html>

Explicación

<programa> ::= <lista expr>

Un programa completo en Ruby (<programa>) está definido ($::=$) como una lista de expresiones (<lista_expr>).

<lista_expr> ::= <expr> [<terminador> <lista_expr>]

Esta regla es recursiva (se llama a sí misma). Nos dice que una "lista de expresiones" arranca con una sola expresión (<expr>).

Los corchetes [] indican que lo que está adentro es **opcional**. Por lo tanto, después de esa primera expresión, *opcionalmente* puede haber un terminador (como /n o ;) seguido de **otra** <lista_expr>. Esto es lo que permite que un programa tenga desde 1 línea de código hasta millones.

```
<expr> ::= <asignacion> | <llamada_metodo> | <estructura_control> |  
          <literal> | <def clase> | <def metodo> | <identificador>
```

Una expresión válida en Ruby puede ser una asignación de variable, o una llamada a un método, o un bloque `if`, o definir una clase, etc. Cualquier línea de código en Ruby encaja en una de estas categorías.

```
<def_clase> ::= "class" <Constante> [ "<" <Constante> ] <terminador>
               <lista_expr>
               "end"
```

Qué significa: Aquí aparecen palabras entre comillas ("`class`" y "`end`"). Estos son los Símbolos Terminales; significa que debes escribir la palabra literalmente así en tu código.

El detalle clave: La parte ["<" <Constante>] es fundamental. Nos dice que, al crear una clase, usar el símbolo < para heredar de otra clase es **opcional** (porque está entre corchetes).

```
<def_metodo> ::= "def" <identificador> [ "(" <lista_params> ")" ] <terminador>
               <lista_expr>
               "end"
```

Qué significa: Se define con la palabra literal "def", seguida de un nombre (<identificador>).

El detalle clave: Fíjate que los paréntesis "(" y ")" están dentro de corchetes []. Esto traduce matemáticamente una característica famosa de Ruby: **los paréntesis al definir métodos son opcionales**.

$$\langle \text{lista_params} \rangle ::= \langle \text{identificador} \rangle [", " \langle \text{lista_params} \rangle] \mid \epsilon$$

Si decides pasar parámetros, defines un nombre (**<identificador>**), y opcionalmente pones una coma seguida de más parámetros.

El detalle clave: El símbolo **ε** (épsilon) en teoría de lenguajes significa **vacío** o "nada". Quiere decir que la lista de parámetros puede ser completamente nula (un método que no recibe argumentos).

```
<estructura_control> ::= "if" <expr> [ "then" ] <terminador>
                        <lista_expr>
                        [ "elsif" <expr> [ "then" ] <terminador> <lista_expr> ]*
                        [ "else" <terminador> <lista_expr> ]
                        "end"
```

Arranca con **"if"**, evalúa una expresión y ejecuta una lista de sentencias.

Al final de la línea del **"elsif"** hay un asterisco *****. El asterisco significa que ese bloque puede aparecer **cero, una, o infinitas veces**. Podés encadenar 50 **elsif** si querés.

La línea del **"else"** solo tiene corchetes **[]** (es opcional), pero **no tiene asterisco**. Esto significa que puede aparecer cero veces o una vez, pero nunca vas a poder poner dos **else** en un mismo bloque lógico.

```
<asignacion> ::= <identificador> "=" <expr>
```

```
<terminador> ::= ";" | "\n"
```

Te dice exactamente cómo la computadora sabe que terminaste de escribir una instrucción. Puede ser con un punto y coma literal **;** o con un salto de línea especial **"\n"** (apretar Enter). Ambas son 100% válidas.

Introducción a Go

Go (frecuentemente llamado Golang) fue concebido en el año 2007 dentro de los laboratorios de Google. Los creadores de este lenguaje son leyendas de la informática: Robert Griesemer, Rob Pike y Ken Thompson (este último, cocreador del sistema operativo Unix y del lenguaje C). El proyecto de código abierto fue anunciado públicamente en noviembre de 2009, y su primera versión estable (Go 1.0) se lanzó en marzo de 2012.

La principal motivación de sus creadores era diseñar un lenguaje que resolviera los problemas de escalabilidad e ingeniería de software masiva que enfrentaba Google. Buscaban combinar la eficiencia, el rendimiento y la seguridad de un lenguaje compilado estáticamente (como C++ o Java), pero con la velocidad de compilación, la simplicidad y la facilidad de lectura de un lenguaje dinámico (como Python o Ruby).

Para lograrlo, tomaron una fuerte inspiración del lenguaje C para la sintaxis básica, de la familia Pascal/Modula/Oberon para el sistema de declaración y paquetes, y de los lenguajes CSP (Communicating Sequential Processes) para diseñar su potente modelo de concurrencia.

Como dato curioso, la mascota oficial del lenguaje es una caricatura de un roedor azul llamado "Gopher", creado por la ilustradora Renée French (esposa de Rob Pike). El Gopher se ha convertido en un ícono tan representativo que a los programadores de Go se los llama informalmente "Gophers".

Características de Go

Tipado Estático y Compilado: A diferencia de Ruby, en Go los tipos de las variables se comprueban durante la compilación. El código escrito se compila directamente a código máquina nativo del sistema operativo, lo que le otorga un rendimiento y una velocidad de ejecución excepcionales.

Enfoque Nativo en la Concurrencia: Es su característica más aclamada. Mediante el uso de "Goroutines" (funciones que se ejecutan de manera concurrente, consumiendo muy poca memoria) y "Channels" (canales para que las Goroutines se comuniquen entre sí), Go hace que aprovechar procesadores multinúcleo sea extremadamente sencillo.

Orientado a Objetos (Sin clases): Go soporta el paradigma orientado a objetos, pero rompe con el modelo tradicional. No utiliza "clases" ni permite la "herencia". En su lugar, utiliza **structs** (estructuras) para agrupar atributos e **interfaces** para definir comportamientos, priorizando la composición por encima de la herencia jerárquica.

Gestión Automática de Memoria: Al igual que Ruby, cuenta con un recolector de basura (garbage collector) para gestionar la memoria. Sin embargo, el de Go está fuertemente optimizado para sistemas en tiempo real, priorizando una latencia ultrabaja para no interrumpir el rendimiento del servidor.

Sintaxis Minimalista y Estricta: Go fue diseñado para tener un conjunto muy reducido de palabras clave (solo 25). Su filosofía es que haya "una sola forma lógica de escribir algo". Además, incluye una herramienta de consola llamada **gofmt** que formatea el código automáticamente, forzando un estilo universal para todos los desarrolladores.

Velocidad de Compilación: La arquitectura de gestión de dependencias (paquetes) de Go fue diseñada desde cero para evitar cuellos de botella. Esto permite que programas con millones de líneas de código se compilen en apenas unos segundos.

BNF de GO

```
<archivo_fuente> ::= <clausula_paquete> ";" { <importacion> ";" } { <declaracion_global> ";" }
<clausula_paquete> ::= "package" <identificador>
<importacion> ::= "import" ( <string> | "(" { <string> ";" } ")" )
<declaracion_funcion> ::=
    "func" <identificador> "(" [ <lista_parametros> ] ")" [ <tipo_retorno> ] <bloque>
<bloque> ::= "{" { <sentencia> ";" } "}"
<sentencia> ::= <declaracion> | <asignacion> | <llamada_funcion> | <estructura_if> | <retorno> | ε
<estructura_if> ::=
    "if" [ <sentencia_simple> ";" ] <expresion> <bloque> [ "else" ( <estructura_if> | <bloque> ) ]
<terminador> ::= ";" | "\n"
```

Explicación

Mientras que Ruby es dinámico y súper flexible, Go (creado por Google) busca ser estricto, predecible y rápido de compilar.

En Ruby se puede empezar a escribir código en la línea 1. Go impone una jerarquía arquitectónica rígida.

`<archivo_fuente> ::= <clausula_paquete> ";" { <importacion> ";" } { <declaracion_global> ";" }`

Qué significa: Esta regla es brutalmente estricta. Te dice que un archivo en Go

(`<archivo_fuente>`) **tiene un orden inamovible:**

1. Primero, obligatoriamente, declararás a qué paquete pertenece (`<clausula_paquete>`).
2. Después, podés tener cero o más importaciones (`{ <importacion> ";" }`).
3. Finalmente, vienen las declaraciones de código como funciones o variables globales (`{ <declaracion_global> ";" }`).

El contraste: En Ruby, un programa era simplemente una `<lista_expr>`. Podías empezar a programar en la línea 1. Go te obliga a organizar la arquitectura del proyecto antes de escribir lógica.

`<clausula_paquete> ::= "package" <identificador>`

`<importacion> ::= "import" ( <string> | "(" { <string> ";" } ")" )`

Qué significa: Para definir el paquete, usás la palabra literal `"package"` seguida de un nombre.

El detalle clave: En `<importacion>`, los paréntesis `( )` funcionan como agrupación lógica. Te dice que podés importar una sola cosa (`"import" <string>`) **O** `(|)` usar paréntesis literales `"( " ... ")"` para importar múltiples librerías de una sola vez, repitiendo el `<string>` con las llaves `{ }`.

`<declaracion_funcion> ::=`

`"func" <identificador> "(" [ <lista_parametros> ] ")" [ <tipo_retorno> ] <bloque>`

Qué significa: Toda función arranca con la palabra `"func"`, seguida de su nombre y paréntesis literales obligatorios `"( " y ")"`.

El contraste: A diferencia de Ruby (donde los paréntesis eran opcionales), acá son estrictamente necesarios. Además, como Go es de tipado estático, aparece el `<tipo_retorno>`, que indica qué tipo de dato devuelve la función.

`<bloque> ::= "{" { <sentencia> ";" } "}"`

Qué significa: Un bloque de ejecución es una repetición de sentencias `{ <sentencia> ";" }`, pero fijate bien cómo están encerradas: usan llaves literales `"{" y "}"`.

El contraste: Acá está la diferencia visual más grande con Ruby. Go abandona la palabra `"end"` y adopta las llaves clásicas de la familia C/C++/Java para delimitar dónde empieza y termina el alcance de las variables.

`<sentencia> ::= <declaracion> | <asignacion> | <llamada_funcion> | <estructura_if> | <retorno> | ε`

Qué significa: Muy similar a la `<expr>` de Ruby. Una instrucción en Go puede ser declarar una variable, asignar un valor, llamar a una función, etc. El símbolo `ε` (épsilon) indica que también puede ser una sentencia vacía.

`<estructura_if> ::=`

`"if" [ <sentencia_simple> ";" ] <expresion> <bloque> [ "else" ( <estructura_if> | <bloque> ) ]`

Qué significa: Arranca con la palabra `"if"`. Luego evalúa una `<expresion>` y si es verdadera, ejecuta un `<bloque>` (que, según la regla 4, implica usar llaves `{ }`).

El detalle clave: La parte [`<sentencia_simple> ";"`] es interesantísima. Te permite ejecutar una instrucción rápida (como crear una variable) en la misma línea del `if`, justo antes de evaluar la condición. Es un patrón súper idiomático en Go.

Además, en el `"else"`, podés concatenar otro `if` (`<estructura_if>`) o simplemente abrir otro `<bloque>`. No existe la palabra `elsif` como en Ruby.

`<terminador> ::= ";" | "\n"`

Qué significa: Exactamente igual que en Ruby. Aunque en la teoría de la EBNF de Go veas un montón de `";"`, en la práctica el compilador de Go inserta los puntos y comas automáticamente cuando detecta un salto de línea `"\n"`.

Algoritmos de Ordenamiento

Ruby: La flexibilidad y el enfoque híbrido (**Array#sort**)

En Ruby, cuando llamas al método `.sort` sobre un arreglo, estás delegando el trabajo pesado a la implementación interna del lenguaje (escrita en C si usamos el estándar MRI - Matz's Ruby Interpreter).

Históricamente, Ruby utilizaba una variante de *QuickSort*. Sin embargo, las implementaciones modernas de lenguajes de alto nivel suelen apoyarse en algoritmos híbridos derivados de **MergeSort** o variaciones similares a **TimSort** para el ordenamiento de arreglos con objetos complejos.

Este enfoque prioriza la **estabilidad**. Un algoritmo de ordenamiento es estable si garantiza que dos elementos con el mismo valor mantendrán el orden relativo original que tenían antes de ordenar. Al trabajar con programación orientada a objetos (donde puedes ordenar por una propiedad específica de un objeto, pero quieres mantener el orden de las demás), la estabilidad es fundamental.

Estos algoritmos están fuertemente optimizados para aprovechar datos que ya tienen un "orden parcial" previo, algo muy común en aplicaciones web o bases de datos de uso real.

Go: La fuerza bruta inteligente (**pdqsort**)

Go tiene un enfoque mucho más ingenieril y enfocado en el rendimiento puro. Hasta la versión 1.18, Go usaba un algoritmo de ordenamiento híbrido estándar, pero en la **versión 1.19** dieron un salto importante al implementar **Pattern-Defeating Quicksort (pdqsort)**.

El Algoritmo: **pdqsort** es un algoritmo híbrido brillante que busca "derrotar los patrones" que normalmente harían que un *QuickSort* tradicional se vuelva lento. Combina::

1. **QuickSort:** Se utiliza como el motor principal porque, en promedio, es el más rápido operando en caché de memoria.
2. **HeapSort:** Si el algoritmo detecta que las particiones de QuickSort están siendo muy ineficientes (el "peor caso" matemático), corta la operación y cambia a HeapSort para garantizar que el tiempo de ejecución no se dispare.
3. **Insertion Sort:** Si el sub-arreglo a ordenar es muy pequeño (típicamente menos de 12 a 24 elementos), cambia automáticamente a Insertion Sort. En arreglos chicos, las iteraciones simples son más veloces que la recursividad compleja.

¿Por qué este enfoque? Go es un lenguaje de sistemas. No asume estabilidad por defecto en su función de ordenamiento principal (aunque ofrece `sort.Stable` si se necesita), priorizando la velocidad máxima de ejecución y el mínimo uso de memoria extra.

¿Qué es `.sort`?

En lenguajes de alto nivel como Ruby o Go, `.sort` (o `sort.Slice` en Go) es simplemente la **interfaz**. Es el comando que el programador escribe. Sin embargo, `.sort` no es un algoritmo en sí mismo; es una "caja negra" que le dice al compilador o intérprete: *"Toma estos datos y ordénalos usando el mejor algoritmo que tengas programado en tu código fuente"*.

Los Algoritmos "Puros"

Los lenguajes modernos casi nunca usan un solo algoritmo de estos, sino que los combinan. Pero hay que entenderlos por separado primero:

- **QuickSort (Ordenamiento Rápido):**
 - **Cómo funciona:** . Elige un elemento del arreglo llamado **pivote**. Luego, reorganiza el arreglo moviendo todos los números menores que el pivote a su izquierda, y todos los mayores a su derecha. Luego repite este proceso recursivamente con la mitad izquierda y la mitad derecha.
 - **Lo bueno:** Es increíblemente rápido en la memoria caché del procesador. Su tiempo promedio es $O(n \log n)$.
 - **Lo malo:** Si el arreglo ya está ordenado o casi ordenado, y el algoritmo elige mal el pivote sistemáticamente, su rendimiento colapsa a $O(n^2)$ (se vuelve lentísimo). Además, **no es estable** (no garantiza mantener el orden original de elementos con el mismo valor).
- **MergeSort (Ordenamiento por Mezcla):**
 - **Cómo funciona:** También usa "Divide y Vencerás". Parte el arreglo a la mitad una y otra vez hasta que quedan sub-arreglos de 1 solo elemento. Luego, empieza a "fusionar" (merge) esos pequeños arreglos, comparándolos y ordenándolos en el proceso, hasta reconstruir el arreglo completo original.
 - **Lo bueno:** Su rendimiento matemático está garantizado, nunca va a ser peor que $O(n \log n)$. Además, es un algoritmo **estable**.
 - **Lo malo:** Necesita crear arreglos temporales en la memoria RAM para hacer las fusiones, por lo que su complejidad espacial es $O(n)$ (consume mucha memoria extra).
- **Insertion Sort (Ordenamiento por Inserción):**
 - **Cómo funciona:** Imagina que estás ordenando cartas en tu mano. Tomas una carta desordenada y la insertas en la posición exacta que le corresponde dentro del grupo de cartas que ya tienes ordenadas.
 - **Lo bueno:** Para arreglos **muy pequeños** (menos de 15-20 elementos), es el algoritmo más rápido que existe, superando a QuickSort y MergeSort porque no requiere la sobrecarga de la recursividad.

Los Algoritmos "Híbridos" (El mundo real de Ruby y Go)

Hoy en día, los lenguajes de programación unen las ventajas de los algoritmos puros creando **híbridos** que se adaptan dinámicamente al tamaño y estado de los datos.

La Implementación en Go: `pdqsort` (Pattern-Defeating Quicksort)

Go (desde la versión 1.19) utiliza este algoritmo híbrido. Es una obra maestra de la optimización que funciona así:

1. **Motor principal:** Arranca usando **QuickSort** porque, estadísticamente, es el más rápido para procesar datos en la memoria caché.
2. **Defensa contra patrones (El colapso):** Mientras hace QuickSort, **pdqsort** cuenta cuántas veces los pivotes están generando malas divisiones. Si detecta que está cayendo en el "peor caso" matemático de QuickSort $O(n^2)$, aborta la misión de inmediato y cambia a **HeapSort** (otro algoritmo puro que garantiza matemáticamente $O(n \log n)$ para mantener el rendimiento).
3. **Para arreglos chicos:** Si durante las divisiones recursivas llega a un sub-arreglo de 24 elementos o menos, abandona QuickSort y aplica **Insertion Sort** para terminar rápido.

La Implementación en Ruby: IntroSort y TimSort

El caso de Ruby es particular porque depende de qué versión o motor estemos analizando (un buen detalle para su investigación):

1. **Ruby Estándar (MRI - Escrito en C):** Su implementación interna de **Array#sort** ha evolucionado hacia un **IntroSort** (Introspective Sort). Funciona casi con la misma filosofía que el **pdqsort** de Go: usa un QuickSort modificado que salta a Insertion Sort para arreglos pequeños y a HeapSort si detecta problemas de rendimiento.
2. **JRuby (Ruby corriendo sobre la máquina virtual de Java):** Aquí Ruby delega el ordenamiento a Java, el cual utiliza **TimSort**. TimSort es un híbrido entre *MergeSort* e *Insertion Sort*. Está diseñado específicamente para aprovechar "secuencias de datos que ya vienen parcialmente ordenadas" en el mundo real. Es súper eficiente para ordenar listas de objetos complejos en aplicaciones web porque garantiza la **estabilidad** (si ordenas usuarios por edad, y dos tienen 20 años, mantienen el orden de registro original).

Resumen Comparativo: Ruby vs. Go (Ordenamiento)

- **Filosofía de Diseño:** Ruby prioriza la previsibilidad y la facilidad para manejar objetos complejos (estabilidad). Go prioriza el rendimiento puro, el bajo nivel y el ahorro de memoria.
- **El Motor Interno (Algoritmo):**
 - **Ruby:** Utiliza algoritmos híbridos (como IntroSort o TimSort, dependiendo del intérprete) que son excelentes para mantener el orden original de datos repetidos.
 - **Go:** Utiliza **pdqsort** (Pattern-Defeating Quicksort), una genialidad técnica que busca la máxima velocidad matemática cambiando de estrategia en tiempo real para no perder eficiencia.
- **Manejo de Memoria (Espacio):** Ruby suele devolver un arreglo completamente nuevo (mayor consumo de RAM). Go muta y reorganiza los datos directamente en la memoria original (*in-place*), lo cual es muchísimo más eficiente.
- **Rendimiento Real (Tiempo):** Como demostramos en el código de prueba, Go (al ser compilado) realiza la misma tarea de ordenamiento en una fracción diminuta del tiempo que le toma a Ruby (lenguaje interpretado).

Ejemplo Ordenamiento en Ruby

```
ordenamientoRuby.rb X
ordenamientoRuby.rb
1  # 1. Ordenamiento básico de un arreglo de primitivos
2  numeros = [9, 2, 5, 1, 7, 5, 15, 9, 20, 21, 1, 2, 3, 0]
3
4  numeros_ordenados = numeros.sort
5  puts "Números ordenados: #{numeros_ordenados}"
6
7  # 2. Ordenamiento de estructuras complejas (Hashes)
8  estudiantes = [
9    { nombre: "Carlos", nota: 9 },
10   { nombre: "Ana", nota: 7 },
11   { nombre: "Belen", nota: 6 },
12   { nombre: "Francisco", nota: 10 },
13   { nombre: "Paula", nota: 7 },
14   { nombre: "Romina", nota: 7 },
15   { nombre: "Jose", nota: 5 },
16   { nombre: "Nicolas", nota: 10 },
17 ]
18
19 # Al ser un algoritmo estable, Las mantendrán su orden original entre ellas.
20 estudiantes_por_nota = estudiantes.sort_by { |estudiante| estudiante[:nota] }
21
22 puts "Estudiantes ordenados por nota:"
23 pp estudiantes_por_nota
```

```
[Running] ruby "c:\Users\NoxiePC\Desktop\SSL\ordenamientoRuby.rb"
Números ordenados: [0, 1, 1, 2, 2, 3, 5, 5, 7, 9, 9, 15, 20, 21]
Estudiantes ordenados por nota:
[{nombre: "Jose", nota: 5},
 {nombre: "Belen", nota: 6},
 {nombre: "Ana", nota: 7},
 {nombre: "Paula", nota: 7},
 {nombre: "Romina", nota: 7},
 {nombre: "Carlos", nota: 9},
 {nombre: "Francisco", nota: 10},
 {nombre: "Nicolas", nota: 10}]

[Done] exited with code=0 in 0.2 seconds
```

Ejemplo Muchos Números para comparar Tiempos en Ruby

You, 5 minutes ago | 1 author (You)

```

1 # 1. Generamos un arreglo con 1 millón de números aleatorios (del 1 al 1000)
2 puts "Generando 1 millón de números en Ruby..."
3 numeros = Array.new(1_000_000) { rand(1..1000) }
4
5 puts "Iniciando ordenamiento..."
6
7 # 2. Capturamos el tiempo de inicio
8 tiempo_inicio = Time.now
9
10 # 3. Ejecutamos el algoritmo interno (TimSort / IntroSort)
11 numeros_ordenados = numeros.sort
12
13 # 4. Capturamos el tiempo final
14 tiempo_fin = Time.now
15
16 # 5. Calculamos la diferencia
17
18 tiempo_total = tiempo_fin - tiempo_inicio
19 puts "Ordenamiento finalizado."
20 puts "-----"
21 puts "Tiempo de ejecución en Ruby: #{tiempo_total} segundos"
22 puts "-----"

```

```
[Running] ruby "c:\Users\NoxiePC\Desktop\SSL\ordenamientoMuchosNumeros.rb"
```

Generando 1 millón de números en Ruby...

Iniciando ordenamiento...

[illegible][illegible]

```
-----
Tiempo de ejecución en Ruby: 0.0745825 segundos
-----
```

```
[Done] exited with code=0 in 0.53 seconds
```

Ejemplo Ordenamiento en Go

```
1  package main
2
3  import (
4      "fmt"
5      "sort"
6  )
7
8  func main() {
9      // 1. Ordenamiento básico de un Slice (arreglo dinámico)
10     numeros := []int{9, 2, 5, 1, 7, 5, 15, 9, 20, 21, 1, 2, 3, 0}
11
12     // sort.Ints modifica el slice original directamente en memoria (in-place)
13     sort.Ints(numeros)
14     fmt.Printf("Números ordenados: %v\n", numeros)
15
16     // 2. Ordenamiento de estructuras complejas (Structs)
17     type Estudiante struct {
18         Nombre string
19         Nota   int
20     }
21
22     estudiantes := []Estudiante{
23         { "Ana", 7 },
24         { "Belen", 6 },
25         { "Francisco", 10 },
26         { "Paula", 7 },
27         { "Romina", 7 },
28         { "Jose", 5 },
29         { "Nicolas", 10 },
30         { "Carlos", 9 },
31     }
32     sort.Slice(estudiantes, func(i, j int) bool {
33         return estudiantes[i].Nota < estudiantes[j].Nota
34     })
35
36     fmt.Printf("Estudiantes ordenados por nota: %v\n", estudiantes)
37     // sort.Slice recibe el slice y una función anónima que define cómo comparar dos elementos.
38     // Nota: sort.Slice usa pdqsort, por lo que NO garantiza estabilidad.
39     // Si se necesitara estabilidad estricta (que Ana siempre quede antes que Belén),
40     // Go requiere llamar explícitamente a otra función: sort.SliceStable()
41 }
```

```
[Running] go run "c:\Users\NoxiePC\Desktop\SintaxisDeParadigmas\01-OrdenamientosRubyGo\ordenamientoGo.go"
Números ordenados: [0 1 1 2 2 3 5 5 7 9 9 15 20 21]
Estudiantes ordenados por nota: [{Jose 5} {Belen 6} {Ana 7} {Paula 7} {Romina 7} {Carlos 9} {Francisco 10} {Nicolas 10}]

[Done] exited with code=0 in 0.388 seconds
```

Ejemplo Muchos Números para comparar Tiempos en Go

```
ordenamientoMuchosNumeroGo.go X
ordenamientoMuchosNumeroGo.go > ...
1 // ordenamiento_tiempo.go
2 package main
3
4 import (
5     "fmt"
6     "math/rand"
7     "sort"
8     "time"
9 )
10
11 func main() {
12     fmt.Println("Generando 1 millón de números en Go...")
13
14     // 1. Generamos un Slice con 1 millón de números aleatorios
15     numeros := make([]int, 1000000)
16     for i := 0; i < 1000000; i++ {
17         numeros[i] = rand.Intn(1000) // Números del 0 al 999
18     }
19
20     fmt.Println("Iniciando ordenamiento...")
21
22     // 2. Capturamos el tiempo de inicio
23     tiempoInicio := time.Now()
24
25     // 3. Ejecutamos el algoritmo interno (pdqsort)
26     sort.Ints(numeros)
27
28     // 4. Calculamos el tiempo transcurrido
29     tiempoTotal := time.Since(tiempoInicio)
30
31     fmt.Println("Ordenamiento finalizado.")
32     fmt.Println("-----")
33     // Usamos %v para imprimir el tiempo formateado automáticamente (ej. 15.4ms)
34     fmt.Printf("Tiempo de ejecución en Go: %v\n", tiempoTotal)
35     fmt.Println("-----")
36 }
37
```

```
[Running] go run "c:\Users\WoxiePC\Desktop\SSL\ordenamientoMuchosNumeroGo.go"
Generando 1 millón de números en Go...
```

Iniciando ordenamiento...

[illegible][illegible]

Tiempo de ejecución en Go: 100.613ms

Fuentes:

Historia de Ruby: <https://www.ruby-lang.org/es/about/>

BNF de Ruby: <https://docs.huihoo.com/ruby/ruby-man-1.4/yacc.html>

Wikibook Ruby: https://es.wikibooks.org/wiki/Programaci3n_en_Ruby

The Go Programming Language Specification: <https://go.dev/ref/spec#Introduction>